Estratégias de testes de **Performance**

"O teste certo para o problema certo."

FÁBIO ARAÚJO

Especialista, Líder de Q.A
Professor e mentor de Qualidade de Software

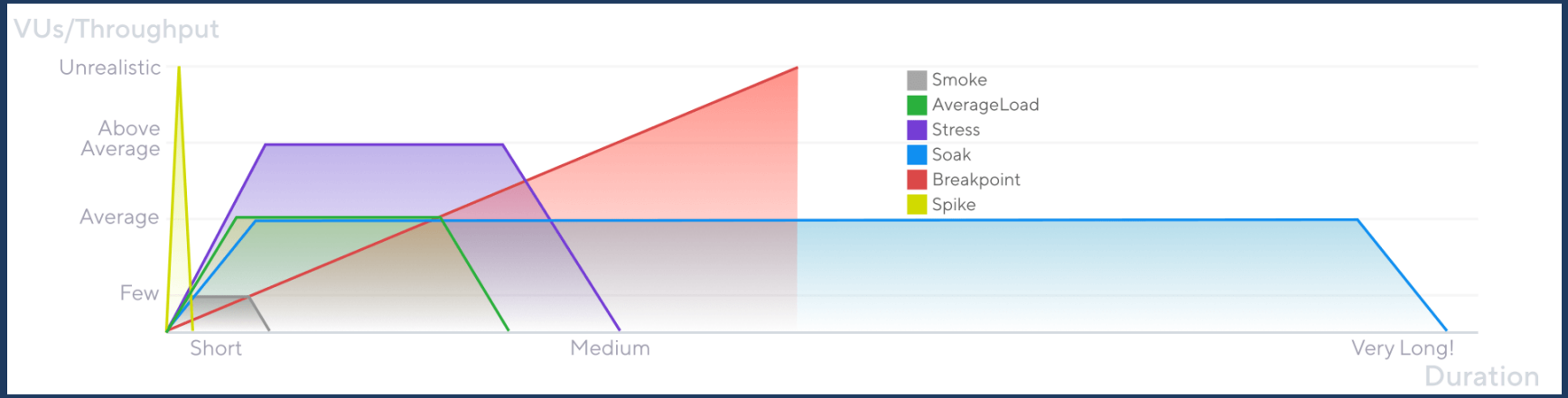
- 26 anos de experiência em TI
- 16 anos dedicados especificamente à qualidade
- Contribuição significativa na cultura de Qualidade, Ágil e DevOps em grandes empresas como Grupo Fiat: Magneti Marelli, **Valor Econômico**, Dotz, Amil, **Ticket**, **Grupo Casas Bahia**, Keeggo e Iguá Saneamento.

O que vamos ver hoje:

Smoke	Load	Stress
Soak	Spike	Breakpoint

Conceito + Cenário Real + Exemplo de Código para cada tipo

Tipos de testes de performance

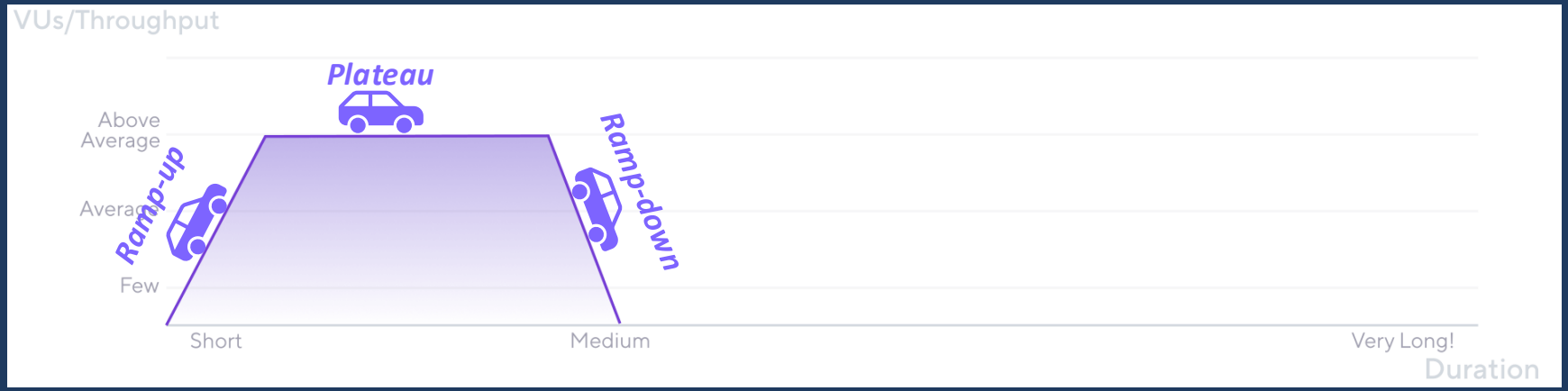


O que vamos ver hoje:

Smoke	Load	Stress
Soak	Spike	Breakpoint

Conceito + Cenário Real + Exemplo de Código para cada tipo

Tipos de testes de performance



Smoke Test

Teste de Fumaça

O que é?

Valida que o script funciona e que o sistema responde corretamente com carga minima

Quando usar?

Sempre que há mudança no código ou no ambiente. É o primeiro teste a rodar antes de qualquer outro.

Objetivo

Detectar erros grosseiros (script quebrado, endpoint indisponível) antes de gastar recursos com testes maiores.

Parâmetros Típicos

VUs / Throughput:	Baixo (1–5 VUs)
Duração:	Segundos a poucos minutos

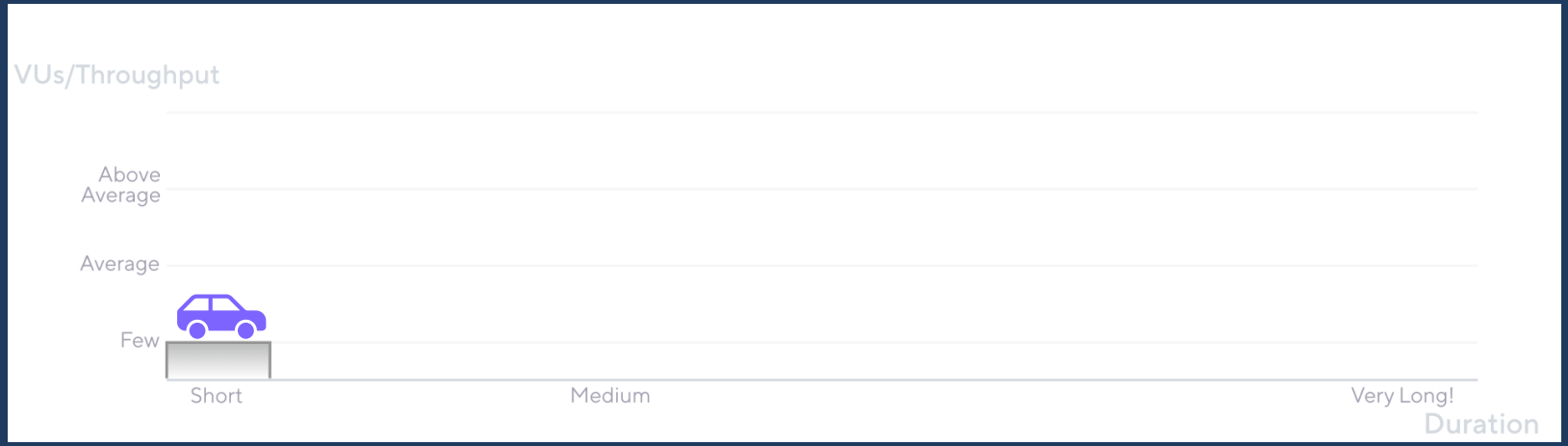
Cenário Real

“Lançamento de uma nova versão da API ou Antes de rodar qualquer teste de carga real mais pesado"

Smoke Test

Teste de Fumaça

Perfil de Carga



Smoke Test: Exemplo de Código

Teste de Fumaça: configuração mínima, validação rápida

script.js

```
1  // smoke.test.js — valida script e resposta básica do sistema
2  import http from 'k6/http';
3  import { check, sleep } from 'k6';
4
5  export const options = {
6    vus: 2,          // apenas 2 usuários virtuais
7    duration: '30s',  // 30 segundos
8    thresholds: {
9      'http_req_failed': ['rate<0.01'], // < 1% de erros
10     'http_req_duration': ['p(95)<200'], // p95 < 200ms
11   },
12 }
13
14 export default function () {
15   const res = http.get('https://api.exemplo.com/produtos');
16   check(res, {
17     'status 200': (r) => r.status === 200,
18     'body não vazio': (r) => r.body.length > 0,
19   });
20   sleep(1);
21 }
```

Average-Load Test

Teste de Carga Média - "O sistema aguenta o uso normal?"

O que é?

Simula a carga real e esperada do sistema em condições normais de produção (tráfego do dia a dia)

Quando usar?

Após mudanças significativas no sistema para confirmar que a performance se mantém estável com o uso médio.

Objetivo

Verificar tempos de resposta, taxa de erros e uso de recursos (CPU, memória) dentro de limites aceitáveis.

Parâmetros Típicos

VUs / Throughput:	Média de produção
Duração:	5 a 60 minutos

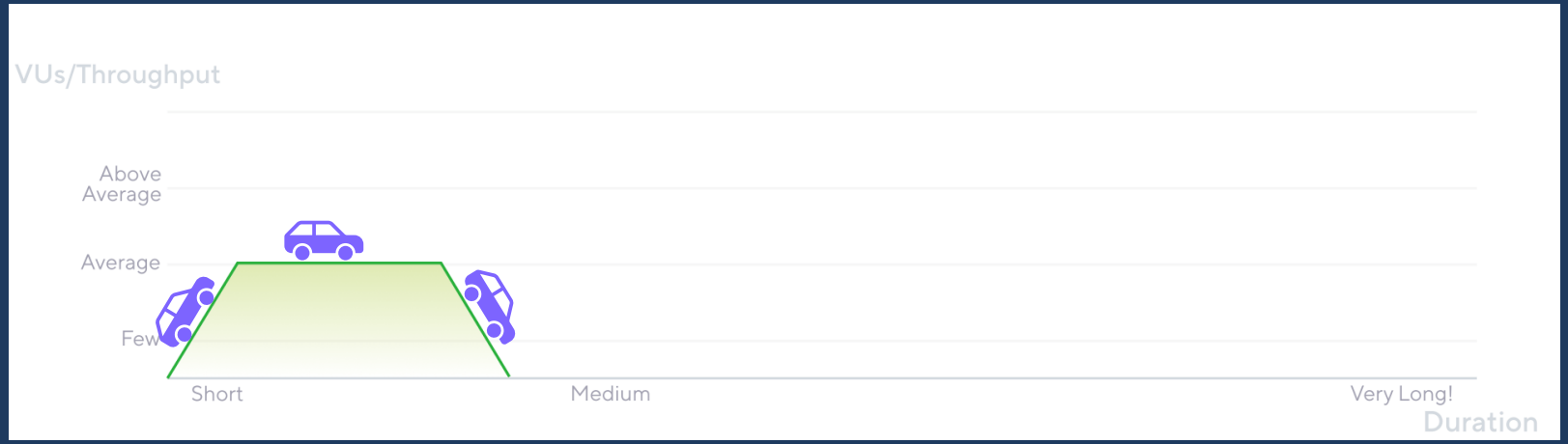
Cenário Real

"O sistema tem em média 500 usuários no pico. Após uma mudança na API de catálogo, roda um Load Test com essa mesma carga por 20 minutos."

Average-Load Test

Teste de Carga Média

Perfil de Carga



Average-Load Test: Exemplo de Código

Carga Média: rampa gradual com plateau na carga de produção

script.js

```javascript
// load.test.js — simula tráfego médio real com stages
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  stages: [
    { duration: '5m',  target: 500 }, // rampa: 0 → 500 VUs em 5 min
    { duration: '20m', target: 500 }, // plateau: mantém 500 VUs por 20 min
    { duration: '5m',  target: 0   },  // rampa: 500 → 0 VUs
  ],
  thresholds: {
    'http_req_duration': ['p(95)<400', 'p(99)<800'],
    'http_req_failed':   ['rate<0.01'],
  },
}

export default function () {
  http.get('https://api.exemplo.com/catalogo');
  sleep(1);
}
```

Stress Test

Teste de Estresse - "Quando ele quebra?"

O que é?

Avalia o sistema com carga **acima** da média esperada, forçando-o além dos limites normais de operação.

Quando usar?

Quando o sistema pode receber picos "previsíveis" (Black Friday, campanhas de marketing, horário de fechamento).

Objetivo

Identificar gargalos, degradação de performance e se o sistema se recupera ao voltar à carga normal.

Parâmetros Típicos

VUs / Throughput:	Alto (acima da média)
Duração:	5 a 60 minutos

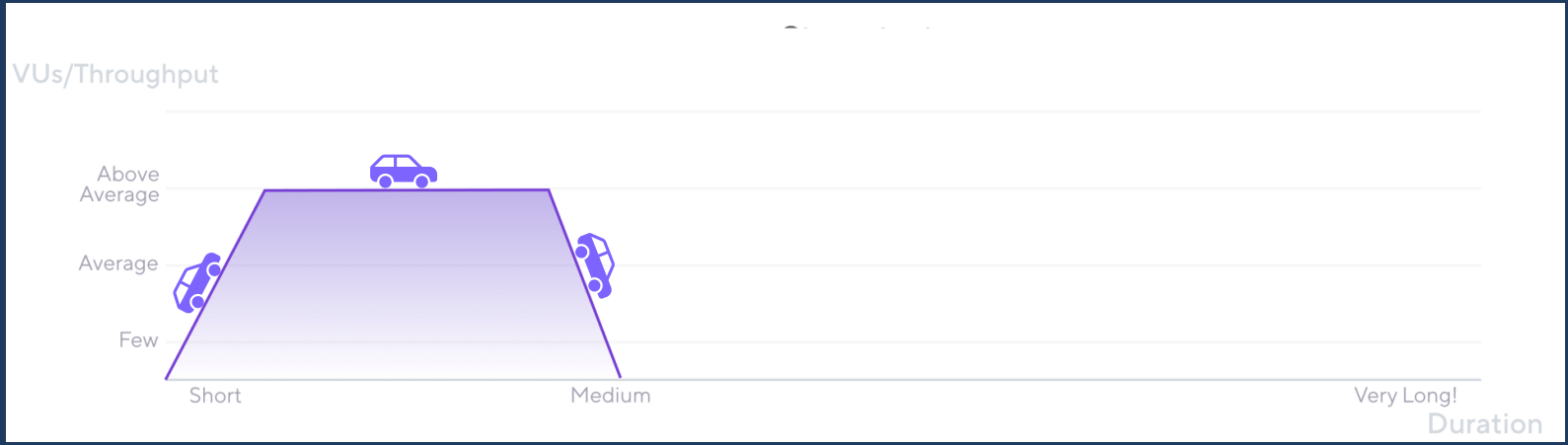
Cenário Real

"Uma fintech sabe que no dia 5 a carga chega a 3x o normal. Antes disso, roda um Stress Test com 1500 usuários por 30 minutos pra identificar o limite e o primeiro ponto de falha."

Stress Test

Teste de Estresse

Perfil de Carga



Stress Test: Exemplo de Código

Estresse: sobe além da média para revelar gargalos

script.js

```javascript
// stress.test.js — 3× a carga média para forçar limites
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  stages: [
    { duration: '5m',  target: 500  }, // carga normal
    { duration: '5m',  target: 1500 }, // 3× — zona de stress
    { duration: '30m', target: 1500 }, // plateau sob stress
    { duration: '5m',  target: 0   },  // recuperação
  ],
  thresholds: {
    'http_req_duration': ['p(95)<1000'], // tolerância maior sob stress
    'http_req_failed': ['rate<0.05'],  // aceita até 5% de erro
  },
}

export default function () {
  http.get('https://api.exemplo.com/pagamentos');
  sleep(1);
}
```

Soak Test

Teste de Imersão - "Ele aguenta ficar rodando por muito tempo?"

O que é?

Avalia a confiabilidade do sistema durante longos períodos de uso contínuo com carga média.

Quando usar?

Após mudanças de infraestrutura, migrações ou refactors para garantir estabilidade prolongada.

Objetivo

Validar a estabilidade do sistema em uso contínuo, identificando: Vazamento de memória, Esgotamento de conexões, Queda progressiva de performance Problemas de gerenciamento de logs, etc.

Parâmetros Típicos

VUs / Throughput:	Média de produção
Duração:	Horas (4h, 8h, 24h)

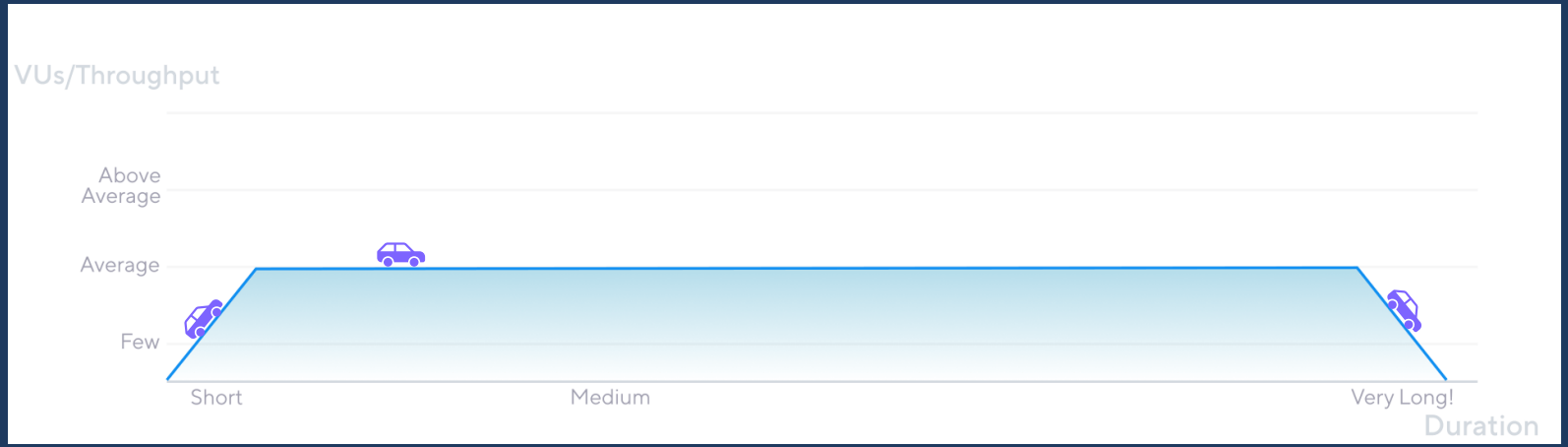
Cenário Real

"Depois de atualizar a versão do Node.js, surgiu uma suspeita de memory leak. Rodamos um Soak Test de 8 horas com 300 usuários, acompanhando pelo Grafana, e vimos a memória crescer continuamente."

Soak Test

Teste de Imersão • também chamado: Endurance Test

Perfil de Carga



Soak Test: Exemplo de Código

Imersão: carga estável por horas para revelar problemas lentos

script.js

```javascript
// soak.test.js — 8 horas de carga contínua (reduza para testes locais)
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '5m',   target: 300 }, // aquecimento
    { duration: '8h',   target: 300 }, // ← plateau longo
    { duration: '5m',   target: 0   }, // resfriamento
  ],
  thresholds: {
    'http_req_duration': ['p(95)<400'],
    'http_req_failed': ['rate<0.01'],
  },
}

export default function () {
  const res = http.get('https://api.exemplo.com/relatorios');
  check(res, { 'ok': (r) => r.status < 400 });
  sleep(1);
}
```

Spike Test

Teste de Pico - “Ele sobrevive a um pico inesperado?”

O que é?

Simula um aumento repentino e massivo de usuários em curtíssimo tempo, o tráfego "viral" ou imprevisto.

Quando usar?

Antes de eventos com tráfego “imprevisível”: lançamentos de produtos, campanhas virais, notícias em destaque.

Objetivo

Verificar se o sistema sobrevive ao pico, como responde sob pressão extrema e se se recupera depois.

Parâmetros Típicos

VUs / Throughput:	Muito alto (5–10× a média)
Duração:	Minutos (pico curto)

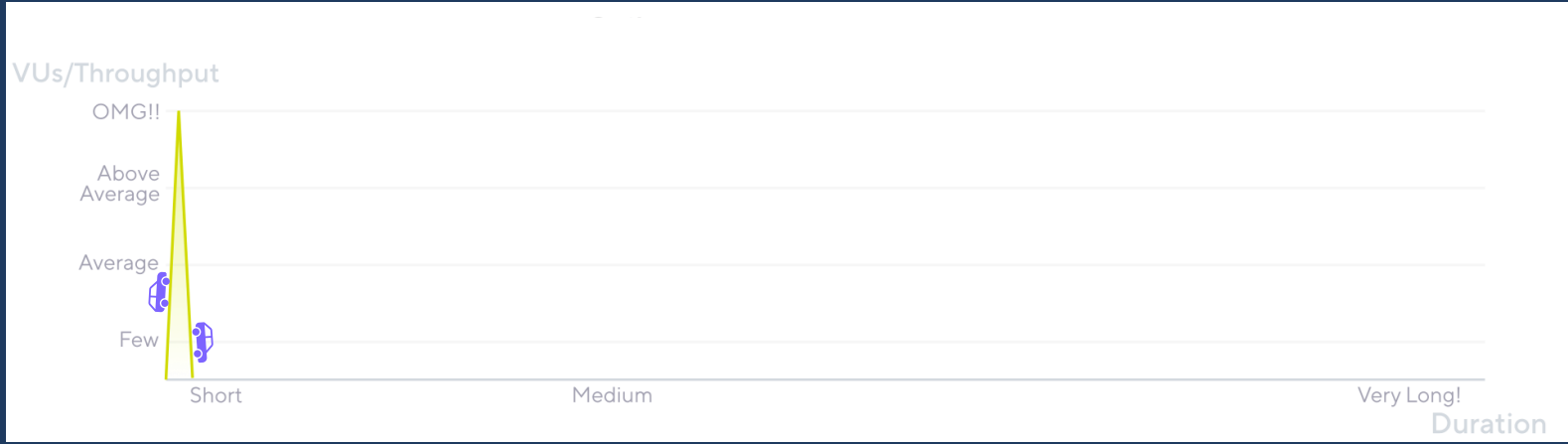
Cenário Real

“Quando abre a venda de um show, milhares de pessoas acessam ao mesmo tempo. O time simula esse pico com 5000 usuários em poucos segundos pra garantir que o sistema não cai e a fila funciona como esperado.”

Spike Test

Teste de Pico

Perfil de Carga



Spike Test: Exemplo de Código

Pico - avalanche repentina de usuários, queda e recuperação

script.js

```
// spike.test.js — simula abertura de vendas / evento viral
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '1m'  target: 100  }, // baseline baixo
    { duration: '10s'  target: 5000 }, // ← spike em 10 segundos!
    { duration: '10s'  target: 100  }, // queda brusca
    { duration: '3m'  target: 100  }, // recuperação?
  ],

  thresholds: {
    'http_req_failed': ['rate<0.10'], // até 10% de erro no pico
  },
}

export default function () {
  const res = http.post('https://api.exemplo.com/ingressos', '{}');
  check(res, { 'compra ok': (r) => r.status < 500 });
  sleep(1);
}
```

Breakpoint Test

Teste de Ruptura - "Qual é o teto real do sistema?"

O que é?

Aumenta a carga gradualmente até o sistema falhar. O objetivo é encontrar o limite máximo absoluto.

Quando usar?

Algumas vezes por ciclo para estabelecer a capacidade máxima, planejar escalabilidade e definir indicadores realistas.

Objetivo

Descobrir: "Até quantos usuários meu sistema aguenta?" e qual componente quebra primeiro (o ponto fraco).

Parâmetros Típicos

VUs / Throughput:	Cresce até a ruptura
Duração:	Enquanto necessário

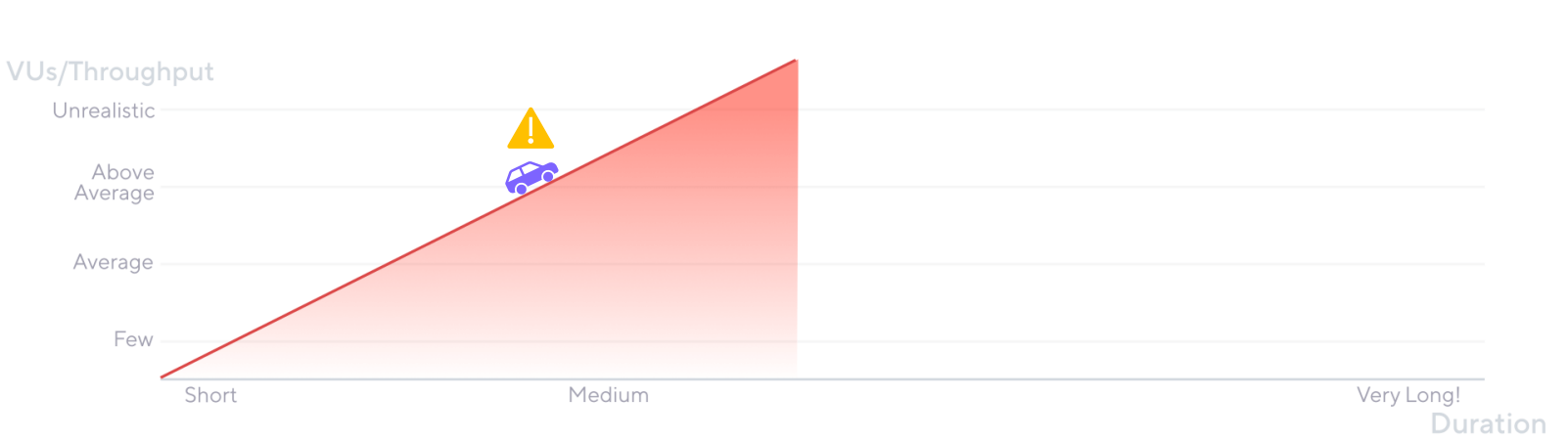
Cenário Real

"Antes da Black Friday, o time decide forçar o sistema de carrinho até o limite. Aumenta a carga gradualmente até cerca de 50 mil usuários e descobre que o banco começa a falhar sob pressão."

Breakpoint Test

Teste de Ruptura

Perfil de Carga



Breakpoint Test: Exemplo de Código

Ruptura: sobe sem parar até o sistema quebrar (use ramping-arrival-rate)

script.js

```javascript
// breakpoint.test.js — aumenta carga gradualmente até o colapso
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  executor: 'ramping-arrival-rate', // controla por req/s, não VUs
  stages: [
    { duration: '2m', target: 200 },  // a cada 2m sobe 200 VUs
  ],
}

export default function () {
  const res = http.get('https://api.exemplo.com/checkout');
  check(res, { 'não quebrou': (r) => r.status < 500 });
}
```

Estratégia Recomendada

Como progredir entre os tipos de teste

1

Smoke Test

Valide o script

2

Avg-Load Test

Baseline normal

3

Stress Test

Acima do normal

4

Soak Test

Estabilidade longa

5

Spike Test

Eventos extremos

6

Breakpoint

Encontre o limite

Obrigado!

linktr.ee/fabio.qa

Referências

- *grafana.com/docs/k6/latest/testing-guides/test-types/*
- *https://grafana.com/docs/k6/latest/testing-guides/*
- *https://quickpizza.grafana.com/*